

SNN 모델을 MCU에서 돌려보자!

정확도 50%, 추론 시간 30s, w/o CMSIS NN

~ 향후 미래 연구와 재미를 위한 시도 ~

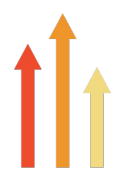
발표자 : 차주형

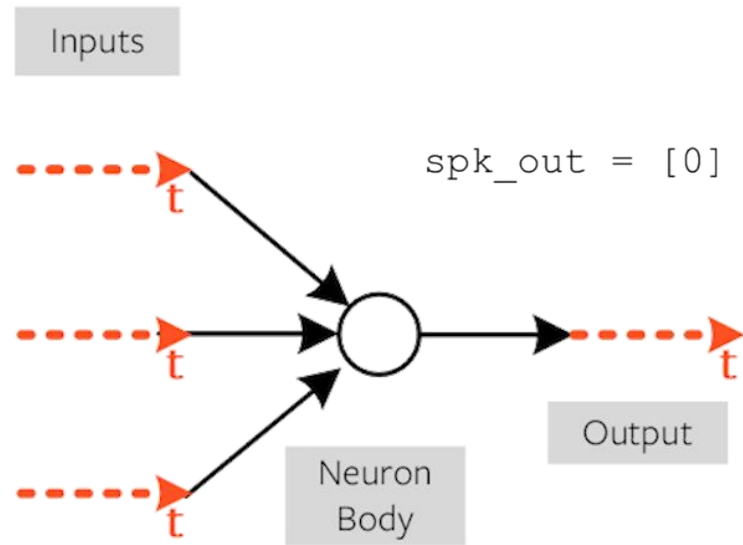


소스코드 위치 : <https://github.com/Piorosen/snn-for-mcu>

1. 채택한, 사용한 SNN 모델의 구조
2. 모델 학습 방법
3. MCU에 배포 방식 (자작 모델 컴파일러)
4. 실험 결과
5. 적용한 최적화 기법들

 PyTorch

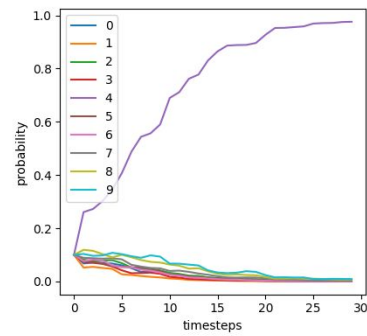
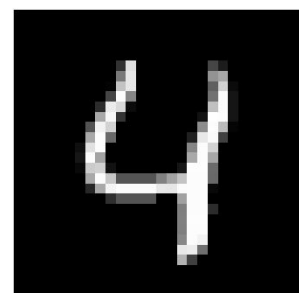
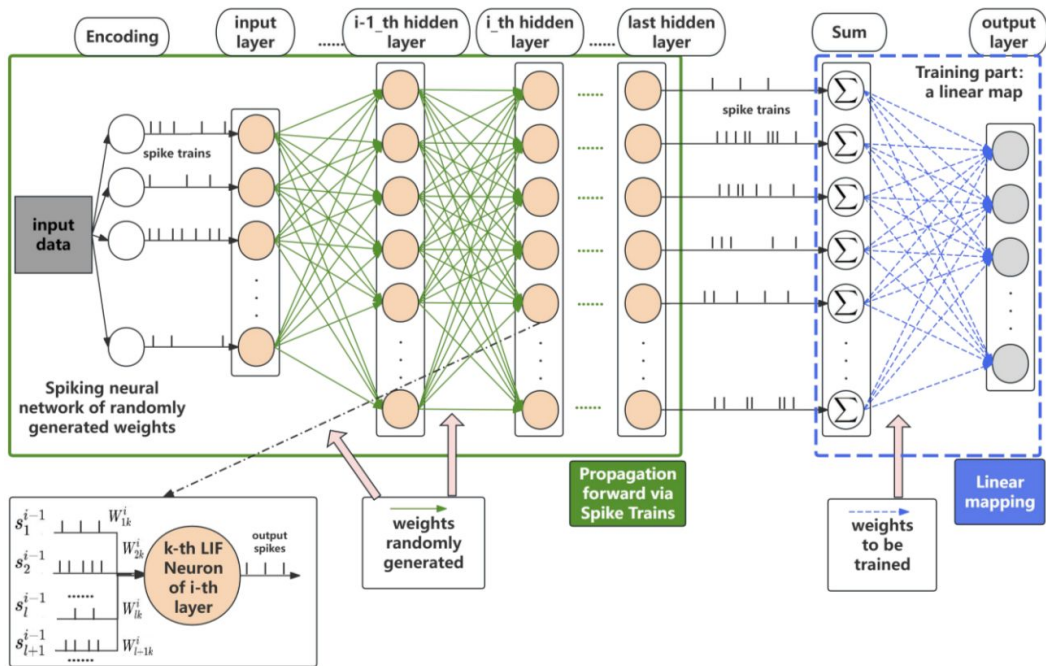
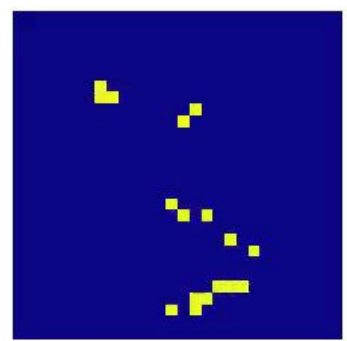
 snnTorch



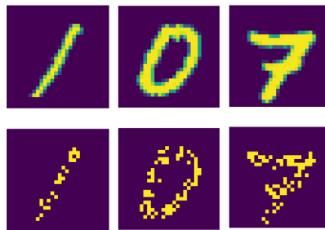
SNN 이 무엇인가?



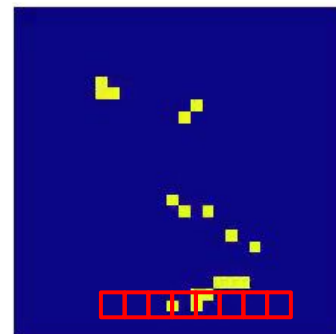
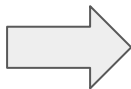
Encoding



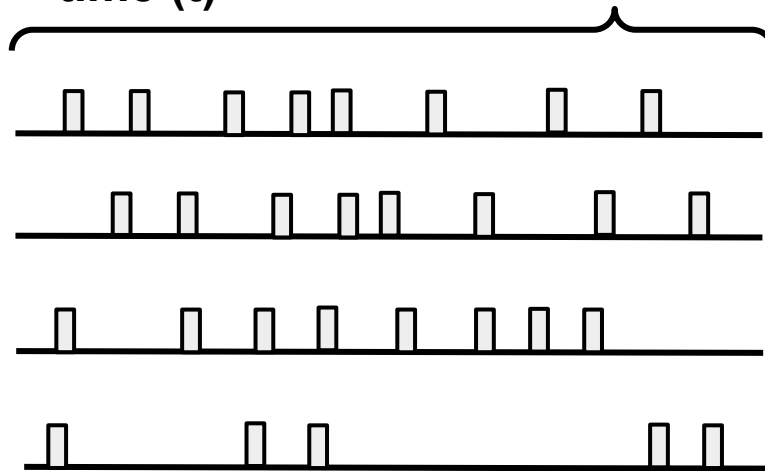
SNN 이 무엇인가?



Encoding

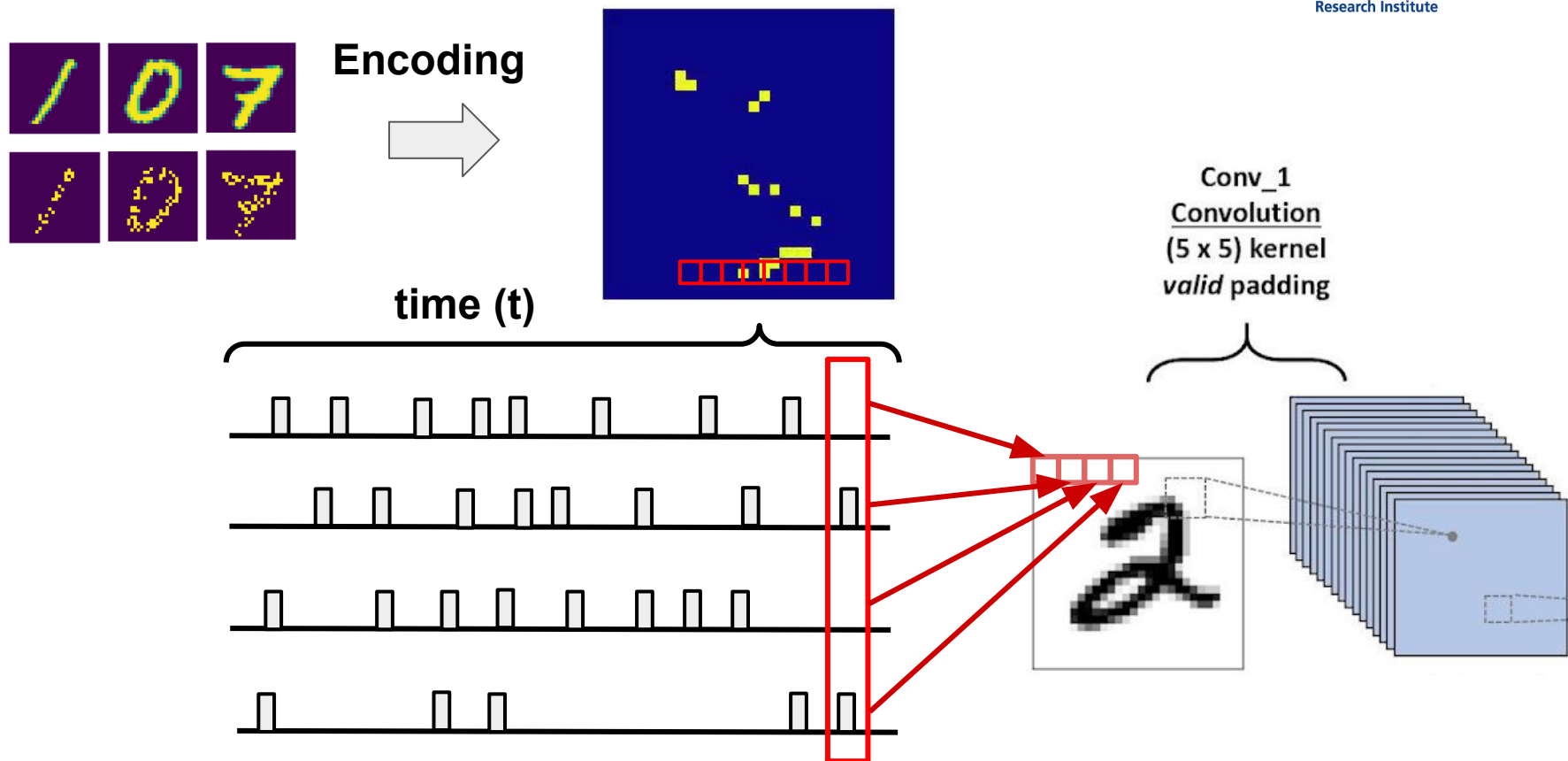


time (t)



해당 픽셀의 값을 의미하고, >>
0 또는 원래 값을 의미함.

SNN 이 무엇인가?

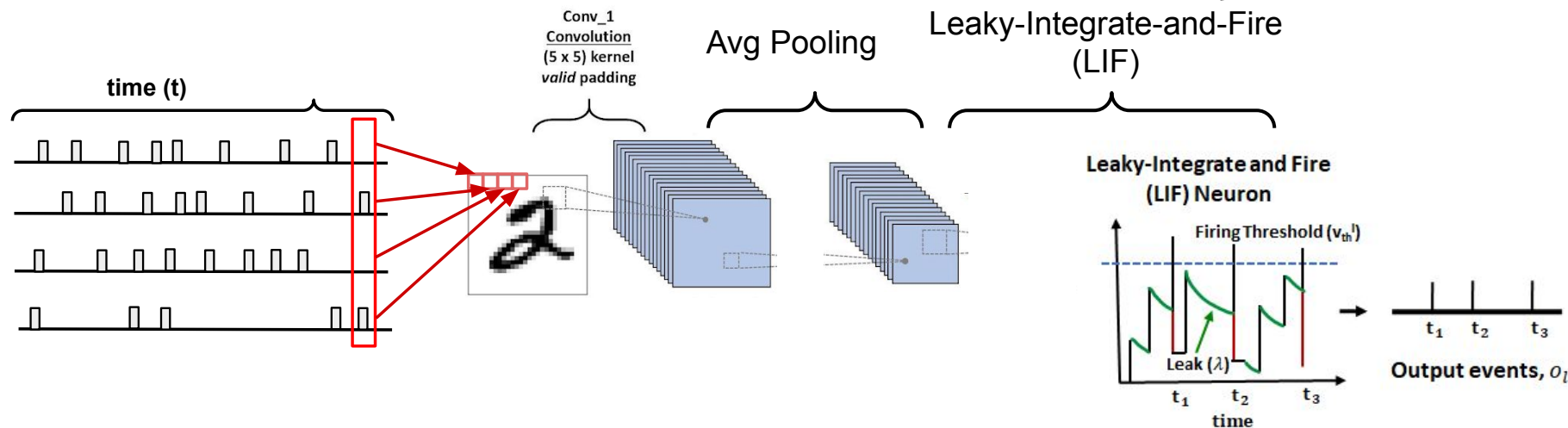


제안 및 사용한 아키텍처 설명

처음부터 모든 것을 다 Spiking 하지는
않다.

1. 인코딩된 값은 입력의 각 요소의 랜덤 값으로 사용함.
2. 활성화 함수에 Spiking 개념을 추가하였음
3. 모델 학습은 LeNet5 기반에 LIF 활성화함수를 활용함.

활성 함수가 Spiking 하게 만들



LeNet5 + LIF 활성화함수 구조



```
net = nn.Sequential(  
    nn.Conv2d(3, 32, 5, padding=2),  
    nn.AvgPool2d(2),  
    snn.Leaky(beta=beta, spike_grad=spike_grad, init_hidden=True),  
  
    nn.Conv2d(32, 64, 5, padding=2),  
    nn.AvgPool2d(2),  
    snn.Leaky(beta=beta, spike_grad=spike_grad, init_hidden=True),  
    nn.Flatten(),  
    nn.Linear(64 * 8 * 8, 10),  
    snn.Leaky(beta=beta, spike_grad=spike_grad,  
              init_hidden=True, output=True)  
).to(device)
```

현재 상태를
저장하기 위함

파이토치에 있는 양자화는 제대로 동작하지 않아, 써드파티 라이브러리를 활용함.

1. 양자화 방식은 QAT 방식으로 수행함.
2. Brevitas 라이브러리로 수행함.
3. Conv, Linear를 대상으로 수행함.

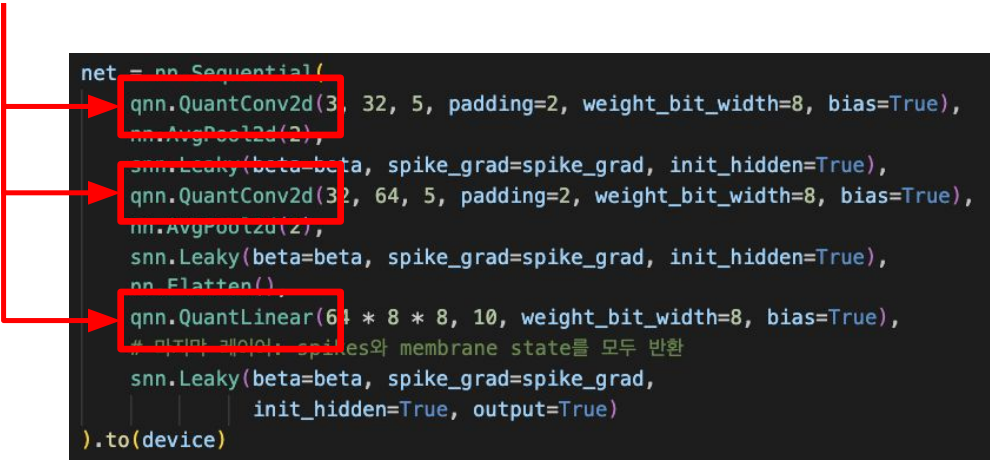
Brevitas

downloads 460k Pytest passing Examples Pytest passing DOI

Brevitas is a PyTorch library for neural network quantization, with support for both *post-training quantization (PTQ)* and *quantization-aware training (QAT)*.

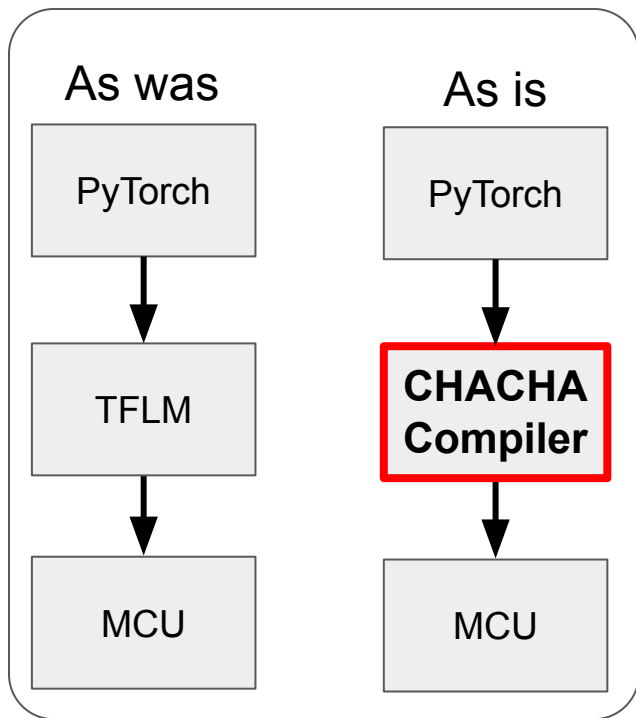
Please note that Brevitas is a research project and not an official Xilinx product.

If you like this project please consider ★ this repo, as it is the simplest and best way to support it.



```
net = nn.Sequential(  
    qnn.QuantConv2d(3, 32, 5, padding=2, weight_bit_width=8, bias=True),  
    nn.AvgPool2d(2),  
    snn.Leaky(beta=beta, spike_grad=spike_grad, init_hidden=True),  
    qnn.QuantConv2d(32, 64, 5, padding=2, weight_bit_width=8, bias=True),  
    nn.AvgPool2d(2),  
    snn.Leaky(beta=beta, spike_grad=spike_grad, init_hidden=True),  
    nn.Flatten(),  
    qnn.QuantLinear(64 * 8 * 8, 10, weight_bit_width=8, bias=True),  
    # 마지막 레이어: spikes와 membrane state를 모두 반환  
    snn.Leaky(beta=beta, spike_grad=spike_grad,  
              init_hidden=True, output=True)  
).to(device)
```

Quantization와 snnTorch를 지원하는 모델 컴파일러가 없어서 만들었음.



컴파일 결과

```
C snn_weights.c
C snn_weights.h
```

.h 파일의 내용

```
1  /* Auto-generated quantized SNN weights (declaration, int-only) */
2  #ifndef CHACHA_SNN_WEIGHTS_H
3  #define CHACHA_SNN_WEIGHTS_H
4
5  #include <stdint.h>
6
7  /* All scales and LIF parameters are Q15 fixed-point (value / 2^15). */
8
9  // Conv1
10 extern const int8_t snn_conv1_weight[32][3][5][5];
11 extern const int16_t snn_conv1_weight_scale;
12 extern const int8_t snn_conv1_bias[32];
13 extern const int16_t snn_conv1_bias_scale;
14
15 // Conv2
16 extern const int8_t snn_conv2_weight[64][32][5][5];
17 extern const int16_t snn_conv2_weight_scale;
18 extern const int8_t snn_conv2_bias[64];
19 extern const int16_t snn_conv2_bias_scale;
20
21 // FC
22 extern const int8_t snn_fc_weight[10][4096];
23 extern const int16_t snn_fc_weight_scale;
24 extern const int8_t snn_fc_bias[10];
25 extern const int16_t snn_fc_bias_scale;
26
```

.cpp 파일의 내용

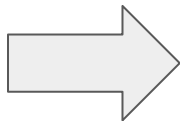
```
1  /* auto-generated SNN weights (definition) */
2
3  #include "snn_weights.h"
4
5  const int8_t snn_conv1_weight[32][3][5][5] = {
6      52, 68, 16, -75, -29, 23, 21, -6,
7      11, 41, 43, 49, -28, 62, -25, -6,
8      4, -74, -28, 51, -62, -32, 22, -1,
9      -27, -5, 25, 68, -42, 14, -75, 5,
10     -48, 12, -21, 22, -62, 53, 72, -1,
11     29, -3, 60, -20, 47, -51, 30, 6, 78, 33, 20, -26,
12     -50, 48, -41, 46, -28, 78, -9, 38, -19, -11, -39,
13     41, -26, -55, 22, 58, 24, -52, -59, 24, -9, -28,
14     -40, -67, -15, 33, -15, -17, -45, 49, -67, -18, -1,
15     41, 46, 61, 17, -69, -15, 10, -102, -87, -35, -7,
```

CHACHA Compiler의 동작


PyTorch

파이토치로
만든 모델

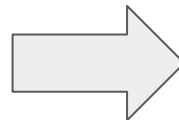
모델 변환 및
검증



 **NumPy**

넘파이로 만든
모델

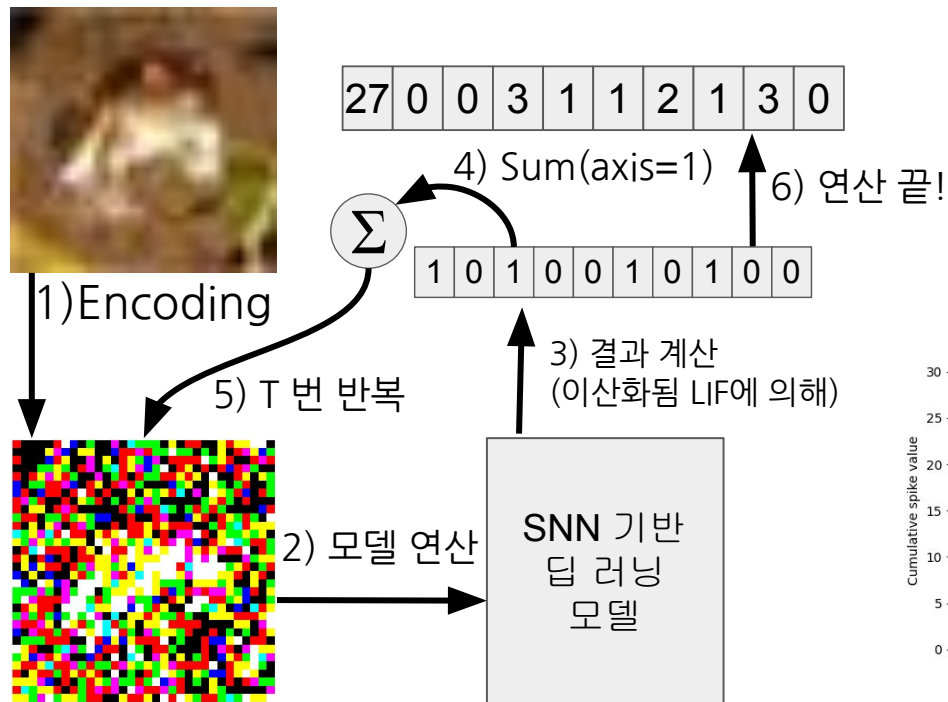
가중치의 값을
C / Header 출력



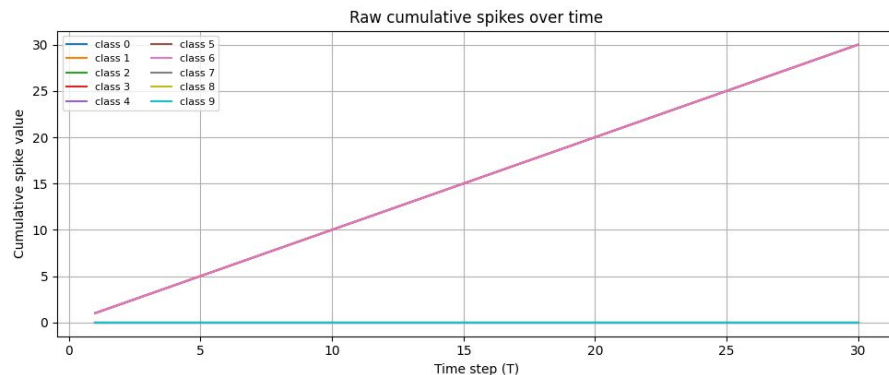
PyTorch 모델을 가중치 파일로 저장하고, MCU 런타임에서 실행하도록 구현함.

시간에 따른, 예측 정확도 변화

매 연산마다 마지막 레이어에서 발생한 스파이크를 집계함.



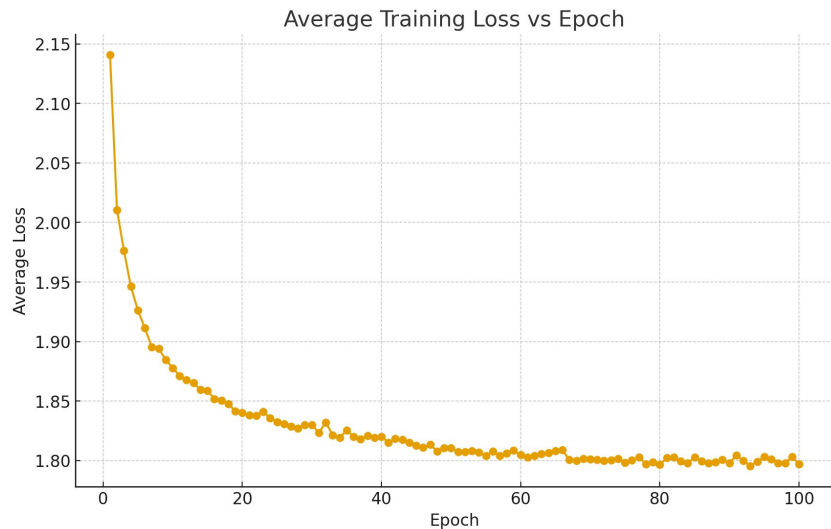
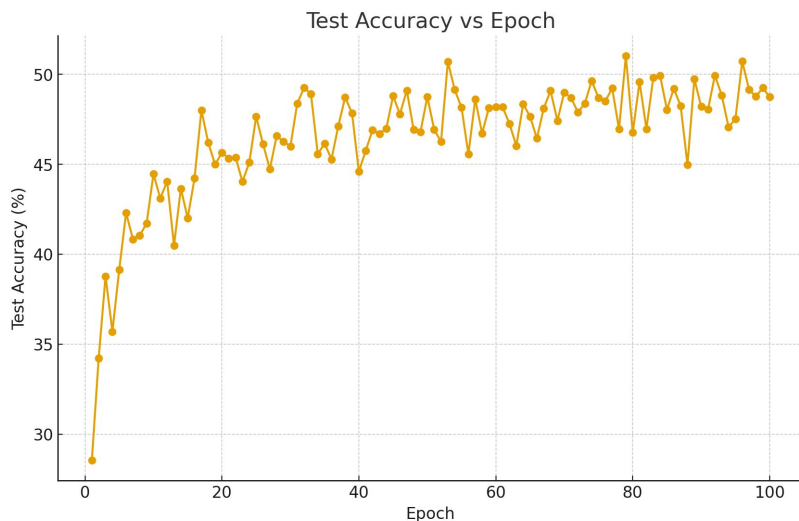
T에 따른 연산 결과 변화



Cifar10의 Loss와 Accuracy 변화 그래프

학습 파라미터 : batch_size = 128, num_epochs = 100, lr = 2e-3

학습 결과 : 양자화된 모델의 경우, 정확도가 50% 한계.



MCU에서의 실행 결과

CHACHA 컴파일러를 통해 생성된 코드의 정확도 분석 (20개 샘플링 후 연산 수행)

```
Final calc = [10.000, 0.000, 0.000, 0.000, 1.000, 0.000, 2.000, 0.000, 27.000, 0.000]
Prediction: answer = 8, class = 8, max_val = 27.000

Final calc = [1.000, 0.000, 0.000, 0.000, 1.000, 0.000, 2.000, 0.000, 27.000, 0.000]
Prediction: answer = 8, class = 8, max_val = 27.000

Final calc = [1.000, 0.000, 27.000, 0.000, 28.000, 0.000, 2.000, 10.000, 0.000, 0.000]
Prediction: answer = 2, class = 4, max_val = 28.000

Final calc = [27.000, 4.000, 0.000, 0.000, 1.000, 0.000, 2.000, 0.000, 0.000, 1.000]
Prediction: answer = 0, class = 0, max_val = 27.000

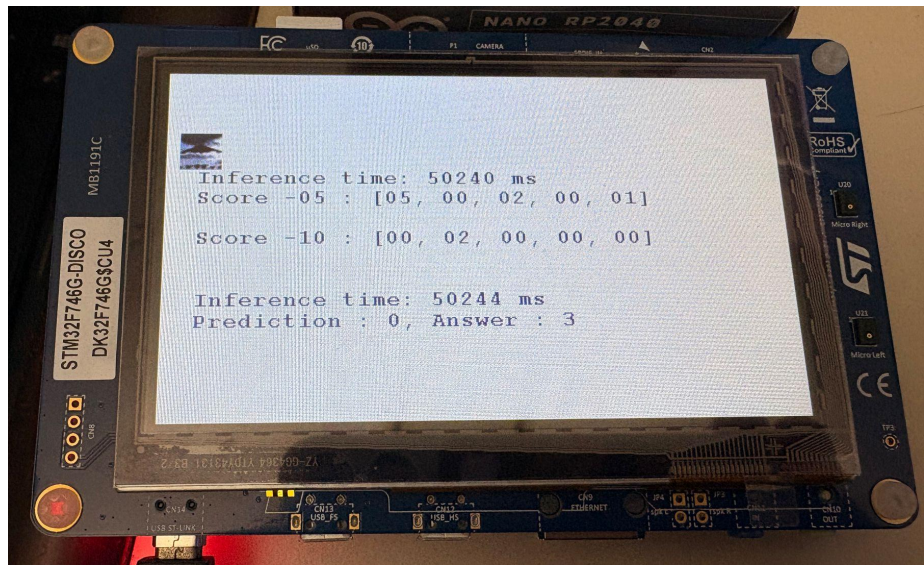
Final calc = [1.000, 0.000, 27.000, 0.000, 28.000, 0.000, 24.000, 0.000, 0.000, 0.000]
Prediction: answer = 4, class = 4, max_val = 28.000

Total samples: 20
Correct samples: 13
Accuracy: 65.00%
```

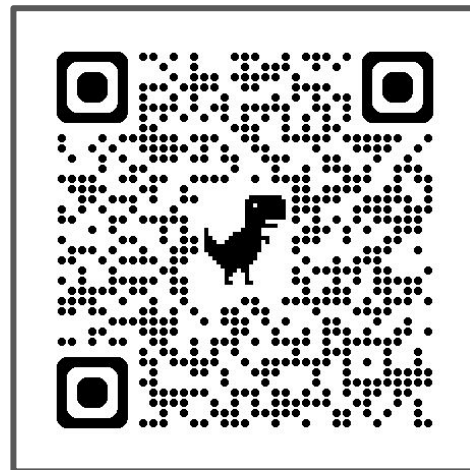

MCU에서의 실행 결과

실제 하드웨어에서의 연산 평가

1. STM32에서 1회 연산에 1.0 초 소요되고, 30번 연산하기 때문에 30초 걸림.
2. CMSIS NN 을 이용한 연산 가속은 수행하지 않았음.
3. 연산 할 수 있겠지만,,,, 더 많은 시간과 추가적인 실험을 한다면, 수행할 계획임.



실행 영상 URL 주소



최적화 방법 (1/2)

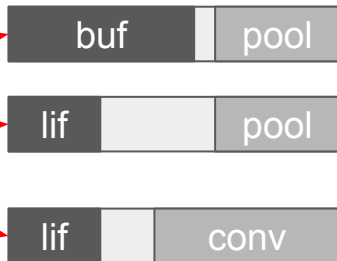
메모리 부족을 해결하기 위한 용량 최적화 여정

Arena 메모리 공간을 활용한
메모리 사용량 최적화

```
13 static int16_t g_lif_out_spk_prev[10];
14
15 // 작업을 버퍼 (중간 feature 공유)
16 // 최대 크기: Conv1 출력 [32][32][32] = 32768
17 #define MAX_FEATURE_SIZE (96 * 32 * 32)
18 static int16_t g_workbuf[MAX_FEATURE_SIZE];
19
20
356 void snn_forward_step(const int16_t* x,
357                      int16_t* spk_out,
358                      int16_t* mem_out) {
359     // g_workbuf 재사용
360     int16_t *buf = g_workbuf;
361
362     // Conv1: 입력 [3][32][32] -> buf[0 .. 32*32*32-1]
363     conv1_forward(x, buf);
364
365     // Pool1: Conv1 출력 [32][32][32] -> pool1_out [32][16][16]
366     int16_t *pool1_out = &buf[MAX_FEATURE_SIZE - 32 * 16 * 16];
367     avgpool2d_2x2_32x32(buf, pool1_out);
368
369     // LIF1: in [32][16][16] -> spk1_out [32][16][16] (buf 앞쪽 재사용)
370     int16_t *lif1_spk = buf; // 크기 32*16*16
371     lif1_step(pool1_out, lif1_spk);
372
373     // Conv2: in lif1_spk [32][16][16] -> conv2_out [64][16][16]
374     int16_t *conv2_out = &buf[MAX_FEATURE_SIZE - 64 * 16 * 16];
375     conv2_forward(lif1_spk, conv2_out);
376
377 }
```

1) 공용 공간 정의

2) 메모리 위치 스왑하면서 공간 사용



const를 이용한 Flash 메모리
오프로딩 기반 최적화

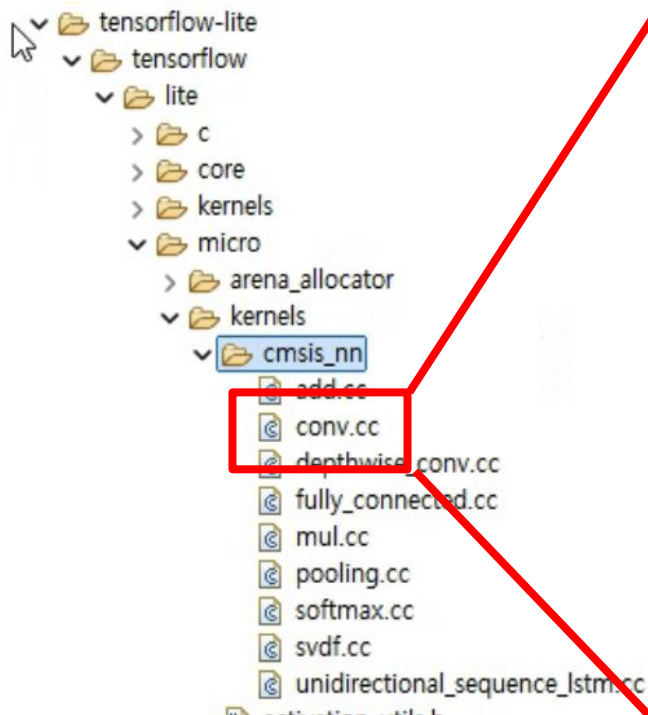
```
const int8_t snn_conv1_weight[32][3][5][5];
const int16_t snn_conv1_weight_scale;
const int8_t snn_conv1_bias[32];
const int16_t snn_conv1_bias_scale;

/2
const int8_t snn_conv2_weight[64][32][5][5];
const int16_t snn_conv2_weight_scale;
const int8_t snn_conv2_bias[64];
const int16_t snn_conv2_bias_scale;

const int8_t snn_fc_weight[10][4096];
const int16_t snn_fc_weight_scale;
const int8_t snn_fc_bias[10];
const int16_t snn_fc_bias_scale;

parameters (Q15 fixed-point)
const int16_t snn_lif1_beta;
const int16_t snn_lif2_beta;
const int16_t snn_lif_out_beta;
const int16_t snn_lif1_threshold;
const int16_t snn_lif2_threshold;
const int16_t snn_lif_out_threshold;
```


향후 계획) CMSIS NN을 이용한 연산 최적화 및 DNN 연산을 TFLite 오프로딩 분석



```
cmsis_nn_conv_params conv_params;
conv_params.dilation.h = params.dilation_height_factor;
conv_params.dilation.w = params.dilation_width_factor;

// Initialize cmsis_nn convolution parameters
conv_params.input_offset = -data.reference_op_data.input_zero_point;
conv_params.output_offset = data.reference_op_data.output_zero_point;
conv_params.stride.h = params.stride_height;
conv_params.stride.w = params.stride_width;
conv_params.padding.h = data.reference_op_data.padding.height;
conv_params.padding.w = data.reference_op_data.padding.width;
conv_params.activation.min = data.reference_op_data.output_activation_min;
conv_params.activation.max = data.reference_op_data.output_activation_max;

// arm_convolve_wrapper_s8 dispatches the optimized kernel accordingly with
// the parameters passed
TFLITE_DCHECK_EQ(
    arm_convolve_wrapper_s8(
        &ctx, &conv_params, &quant_params, &input_dims,
        tflite::micro::GetTensorData<int8_t>(input), &filter_dims,
        tflite::micro::GetTensorData<int8_t>(filter), &bias_dims,
        tflite::micro::GetOptionalTensorData<int32_t>(bias), &output_dims,
        tflite::micro::GetTensorData<int8_t>(output)),
    ARM_CMSIS_NN_SUCCESS);
```

THANK YOU